MERN STACK

TRAINING TR-103 REPORT DAY 10

6July 2026

Overview:

The tenth day of the MERN Stack training focused on advanced JavaScript concepts, including switch statements, loops, break and continue statements, functions, and the Document Object Model (DOM). I learned how to control program flow, repeat tasks efficiently using loops, create reusable functions, and interact with HTML elements using the DOM. Practical exercises helped us understand how JavaScript adds functionality and interactivity to web pages.

Learning Objectives:

- Understand the working of the switch statement.
- Learn different types of loops in JavaScript.
- Understand the use of break and continue statements.
- Learn how to create and use functions.
- Understand the basics of the Document Object Model (DOM).
- Build interactive web pages using JavaScript.

JAVASCRIPT

Switch Statement

The switch statement is used to execute one block of code from multiple options based on the value of an expression. It provides a simpler alternative to multiple if...else statements.

Features:

- Compares a variable with multiple cases.
- Executes the matching case.
- Uses break to exit the switch.
- Uses default if no case matches.

Loops

A loop is used to execute a block of code repeatedly until a specified condition becomes false. Loops help reduce repetitive code and make programs more efficient.

Types of Loops

1. For Loop

The for loop is used when the number of iterations is known in advance.

Features:

- Includes initialization, condition, and update.
- Best for counting and fixed repetitions.

```
console.log("Task : To print number from 1to 25 but skip multiply of 7 : ")
for (let i = 1; i <= 100; i++) {
    if (i % 7 == 0) {
        continue;
    }

    if (i == 25) {
        break;
    }
    console.log(i);

}
```

```
Task : To print number from 1to 25 but skip multiply of 7 :
1
2
3
4
5
6
8
9
10
11
12
13
15
16
17
18
19
20
22
23
24
```

2. While Loop

The while loop executes a block of code as long as the given condition is true.

Features:

- Checks the condition before execution.
- Suitable when the number of iterations is unknown.

```
while (i < 5) {
    console.log(i)
    i++
}
```

```
0
1
2
3
4
5
6
7
```

3. Do-While Loop

The do...while loop executes the code once before checking the condition.

Features:

- Executes at least one time.
- Checks the condition after execution.

```
console.log("DO-WHILE LOOP")
let j=1;
do{
    console.log(j)
    j++;
}
while(j%7 ==0)
```

```
DO-WHILE LOOP
1
2
3
4
5
6
7
8
9
```

Break Statement

The break statement is used to immediately terminate a loop or switch statement.

Uses:

- Exits a loop completely.
- Exits a switch statement after a matching case.

Continue Statement

The continue statement skips the current iteration of a loop and moves to the next iteration.

Features:

- Skips the remaining code of the current iteration.
- Does not terminate the loop.

Functions

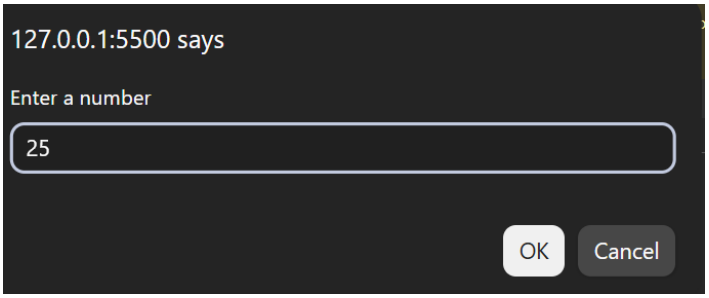
A function is a reusable block of code that performs a specific task. Functions help organize code, reduce repetition, and improve readability.

Features:

- Reusable code blocks.
- Can accept parameters.
- Can return values.

```
let n = parseInt(prompt("Enter a number"))
function check() {
  if(n%2==0)
{
  return "even"
}
else{
    return "odd"
}

}
console.log(check())
```



odd function.html:34

DOM (Document Object Model)

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of elements. JavaScript uses the DOM to access, modify, and update the content, structure, and style of web pages dynamically.

Features:

- Represents the webpage as a tree structure.
- Each HTML element is a node.
- Allows dynamic changes to webpage content and styles.

Task 1 :

```
 <p1>This is HTML</p1><br>
<p2>This is CSS</p2><br>
<p3>This is JS</p3><br>
<button onclick="change()">Click me</button>
 <button onclick="change1()">Click me</button>
 <button onclick="change2()">Click me</button>


<script>
   let p1 = document.getElementsByTagName("p1")[0]
   let p2 = document.getElementsByTagName("p2")[0]
   let p3 = document.getElementsByTagName("p3")[0]
   function change(){
       p1.innerHTML="This is HTML"
       p1.style.display = "none";

   }
   function change1(){
       p2.innerHTML="This is CSS"
       p2.style.display = "none";

   }
   function change2(){
       p3.innerHTML="This is JS"
       p3.style.display = "none";

   }
```



Task 2 :

```
<p class="inc">0</p>
<button onclick="incr()">INC</button>

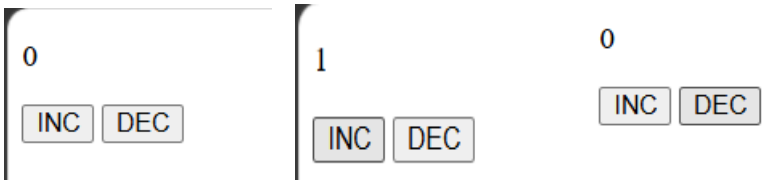
<button onclick="dec()">DEC</button>

<script>

    let p = document.getElementsByTagName("p")[0]
    let count = 0;
    function incr(){
        count++;
        p.innerHTML = count;
    }
    function dec(){
        count--;
        p.innerHTML = count;
    }

   let t= document.getElementsByTagName("p")[0]
```

Output:



Conclusion:

By the end of Day 10, I gained a better understanding of JavaScript control statements, loops, functions, and the DOM. These concepts helped me write more efficient programs and understand how JavaScript interacts with HTML to create dynamic and interactive web pages.